ChatGPT

Experimental Design for a Dual-Corpus Evaluation of a Browser Performance Profiler

A strong methodological precedent exists for combining **controlled, manipulable Web workloads** with **real or real-derived Web pages**, but the two corpora answer different questions and should not be treated as interchangeable evidence. Web-performance research has repeatedly moved between synthetic pages, replayed production pages, and live/popular sites: WProf evaluated an in-browser profiler on hundreds of pages; the SPDY study explicitly progressed from simple synthetic pages to complete loads based on popular sites; Mahimahi recorded real HTTP applications and replayed them under controlled network conditions; and current browser benchmarks such as Speedometer use deliberately constructed application workloads designed to exercise different browser subsystems. [1]

The resulting methodological case for your proposed architecture is therefore quite strong: **controlled fixtures can establish causal sensitivity and attribution against known perturbations, while a separate live-site corpus can establish behavior under production complexity and variability**. Those are complementary validity arguments rather than two versions of the same experiment. The synthesis below should be treated as a set of defensible design options, not as a final protocol, sample size, hypothesis, or research gap.

Literature designs and experimental terminology

How relevant studies construct Web-performance corpora

There is no single standard corpus construction pattern in Web-performance research. Instead, the corpus tends to follow the claim being made.

Study or benchmark	Corpus and experimental design	Methodological relevance to your profiler evaluation
WProf — Wang et al., NSDI 2013	WProf is an in-browser page-load profiler that constructs dependency graphs spanning networking, parsing, JavaScript/CSS evaluation, and rendering. Its evaluation studied **350 Web pages** under conditions including caching, SPDY, and server-side optimization. The detailed methodology used popular Alexa pages plus an additional random sample from the broader Alexa population, and performed repeated loads; some controlled protocol experiments copied page resources to a controlled server. [2]	Particularly close to your problem because the research object was itself a browser profiler. It demonstrates that profiler evaluation need not choose between production pages and controlled experiments.

Study or benchmark	Corpus and experimental design	Methodological relevance to your profiler evaluation
How Speedy is SPDY? — Wang et al., NSDI 2014	The authors deliberately progressed from **simple synthetic pages to complete page loads based on the Alexa top 200**, isolating factors such as network properties, object number/size, computation, and page dependencies before reintroducing realism. [3]	This is perhaps the clearest precedent for a staged "understand factors under control, then test realistic pages" methodology.
Mahimahi — Netravali et al., USENIX ATC 2015	Mahimahi records HTTP application traffic and later replays it under emulated network conditions while preserving the multi-server structure of Web applications. The paper notes that multi-server loading occurred in 98% of the Alexa US Top 500 pages studied and evaluated protocols over a 500-site corpus. [4]	Provides an intermediate point between hand-built fixtures and uncontrolled live traffic: **real-derived but replayable workloads**.
Understanding Website Complexity — Butkiewicz et al., IMC 2011	Rather than relying only on informal labels, the study characterized pages using measurable **content-level features**, such as number and size of images, and **service-level features**, including the servers and administrative origins involved in loading a page. It also compared popularity ranges and website categories. [5]	Strong precedent for defining categories using observable workload properties rather than subjective labels such as "heavy."
Klotski — Butkiewicz et al., NSDI 2015	The evaluation included landing pages randomly selected from a defined popularity range—for example, 50 pages selected from the Alexa top 200 for much of the experimental work. [6]	Shows a useful distinction between a **defined sampling frame** and random selection inside it.
Polaris — Netravali et al., NSDI 2016	Recorded 200 sites from the Alexa Top 500 using Mahimahi and included several workload categories such as news, e-commerce, and social media. [7]	Illustrates popularity filtering plus purposive diversity of application categories.
Speedometer 3.1	The browser benchmark runs simulated interactions over deliberately selected application workloads including to-do applications implemented in multiple frameworks, news rendering, charting, code editing, and other application patterns. [8]	A current example of **controlled application workloads chosen to exercise distinct browser behaviors**, rather than attempting to sample the Web statistically.

Study or benchmark	Corpus and experimental design	Methodological relevance to your profiler evaluation
Zeber et al., WWW 2020	Compared repeated automated Web crawls with an opt-in dataset representing more than 50,000 Firefox users, while separately examining time, cloud versus residential IP addresses, and operating-system effects. The authors found meaningful differences between stateless automated crawling and stateful human browsing. [9]	Direct warning that a synthetic browser agent cannot automatically be equated with the experience of a human user.
Jueckstock et al., WWW 2021	Used synchronized visits to the same URLs across combinations of **three network vantage points and two browser configurations**, holding other factors such as workflow, DNS, entropy sources, bandwidth, and latency as constant as practicable. The site frame was a frozen Tranco top-25K snapshot. [10]	Very relevant to your configurable synthetic-agent concept: agent configuration and vantage point can themselves be experimental variables.
HTTP Archive / Web Almanac	The 2025 methodology used over 16 million sites sourced from CrUX, tests from datacenter infrastructure, separate desktop/mobile configurations, empty-cache logged-out loads, and increasingly includes secondary pages because the authors explicitly note that a homepage alone may not represent an entire site. [11]	Strong large-scale precedent, but also an unusually clear statement of the ecological limitations of lab-style crawling.

The literature therefore contains at least four recognizable corpus types: **synthetic fixtures**, **standardized application benchmarks**, **record-and-replay captures of real pages**, and **live/popularity-derived Web corpora**. The strongest analogy to your proposal is not any one paper but the methodological progression in the SPDY/WProf/Mahimahi line of work: isolate causes under control, then restore realistic Web structure and variability. [12]

Terminology that will keep the study precise

Some terms that sound interchangeable have different methodological implications.

Term	Recommended usage in this study
Baseline	A designated reference implementation or configuration against which differences are expressed. A baseline is not automatically a statistically valid “control”; it simply provides the reference point.

Term	Recommended usage in this study
Control condition	The condition that does **not receive the manipulation of interest** but otherwise undergoes the same experimental procedure. For a controlled fixture, the unperturbed version can legitimately be the control condition. A sham rebuild or redeployment with no defect added can be an even stronger procedural control.
Treatment / perturbed condition	A condition in which an experimenter deliberately manipulates a factor—for example, increasing image payload while trying to preserve other page characteristics. “Perturbation” is particularly natural terminology for profiler sensitivity experiments.
Benchmark	The standardized **evaluation procedure or suite**, not necessarily an individual page. Speedometer, for example, calls itself a browser benchmark and contains multiple workloads. [13]
Workload	The work presented to the browser: page/application content plus, where relevant, the interaction sequence and inputs. Speedometer explicitly distinguishes its benchmark from the individual application workloads that implement user journeys. [13]
Fixture	A useful software-testing term for a deliberately maintained test site/application and its assets. “Controlled benchmark fixture” is reasonable, although “controlled workload fixture” is more precise when the fixture itself is not intended to constitute an entire standardized benchmark.
Corpus	The collection of sites, pages, applications, or captured workloads included in the evaluation. Mahimahi and later Web-performance studies use collections of hundreds of sites in this sense. [14]
Agent profile	A reproducible client-side configuration representing a device/network/browser/user-condition scenario. It is preferable to “user” when no actual human behavior is being observed.

A further distinction is important for the live corpus: a live production page is usually **an observational workload**, not a treatment. Calling a live site the “control” merely because it has not been deliberately modified would be misleading. Treatment/control terminology is most natural in the manipulable fixture tier or when a genuine intervention—such as profiler enabled versus disabled—is applied to the live page.

Controlled-fixture design and factor manipulation

A useful sensitivity-oriented structure

For the controlled tier, a particularly defensible structure would be:

baseline fixture → one-factor perturbation families → dose levels → replicated executions → optional interaction experiments.

That mirrors the logic of the SPDY study, which first isolated individual determinants before incorporating page dependencies and realistic computation, and it accords with general design-of-experiments practice in which factors are intentionally varied while nuisance sources are controlled or randomized. [15]

The strongest version of such an experiment would preserve not only the source repository but also **explicit ground truth for the injected defect**. The profiler can then be evaluated against something known independently of the profiler itself.

Perturbation family	Possible controlled manipulation	Ground truth worth preserving	Main isolation problem
Large image payload	Replace an image with visually equivalent variants having progressively greater encoded size while preserving layout dimensions. A separate family could vary decoded pixel count while keeping transfer bytes approximately controlled.	Encoded bytes, dimensions, decoded pixel area, request timing, compression/ format.	“Large image” is not one mechanism: transfer, decode, rasterization, memory, and paint costs may all move together.
Heavy JavaScript transfer	Increase downloaded JavaScript while keeping executed work relatively constant.	Script transfer bytes, script count, compressed/ uncompressed size.	Otherwise transfer cost becomes confounded with parse/compile/ execution cost.
Heavy JavaScript execution	Add deterministic or calibrated computation independently of script transfer size.	Actual main-thread CPU duration and call/task structure measured independently of the profiler under evaluation.	Runtime varies with CPU, browser optimization, JIT state, and thermal state.
Long main-thread tasks	Introduce scheduled work intended to occupy the main thread for controlled durations.	Start/end timestamps and intended/ observed task duration.	The W3C Long Tasks API uses **50 ms or more** as the threshold for a long task, so perturbations near that boundary require particular care. [16]
Large DOM	Replicate hidden or visually equivalent structural elements while holding payload and presentation as constant as practical. Independently vary node count, depth, or branching if those distinctions matter.	Exact node count, depth, element types, mutation history.	DOM growth can also increase HTML bytes, style matching, layout, memory use, and accessibility-tree work.

Perturbation family	Possible controlled manipulation	Ground truth worth preserving	Main isolation problem
Many requests	Split a fixed total payload into more resources rather than simply adding more bytes. A complementary experiment can increase requests and bytes together.	Request count, total bytes, per-resource sizes, origins, protocol.	HTTP/1.1, HTTP/2, and HTTP/3 respond differently to request multiplicity, so protocol cannot be silently allowed to drift.
Layout instability	Insert or resize content after rendering in a controlled viewport, with a known geometry and timing.	Individual layout-shift entries, affected regions and expected score components.	CLS is defined over unexpected shifts and session windows, so a single injected movement and whole-page CLS are not identical constructs. [17]
Resource inefficiency	Treat individual inefficiencies separately: disabled compression, missing caching, duplicated downloads, render-blocking resources, oversized imagery, and so forth.	Exact changed headers/assets/ dependencies and resulting byte/ request properties.	"Inefficient" is too broad to constitute one causal factor.
Third-party dependency load	Introduce a controlled mock third-party origin or increasing numbers of independent origins/services.	Origin count, request count, bytes and artificial response delays.	Real third parties introduce additional, uncontrolled server and personalization behavior.

The critical design principle is that **"one factor at a time" refers to the deliberately manipulated cause, not necessarily to one and only one downstream metric changing**. Increasing image bytes can change network completion time, decoding, memory pressure, paint scheduling, and perhaps LCP. Those are plausible mediators, not necessarily design failures. The important requirement is that unrelated source differences do not unintentionally accompany the perturbation.

This is why a source-level **perturbation manifest** would be valuable. For every fixture build, it could state:

> Parent baseline commit → factor changed → factor level → exact resource/code modification → expected mechanical consequence → quantities known independently of profiler output.

That gives the evaluation something closer to a calibration artifact.

Prefer dosage curves to a single "bad" variant when sensitivity matters

A binary baseline-versus-bad comparison can establish that the profiler reacts to one perturbation magnitude. It does not reveal whether its reported signal scales sensibly with the underlying defect.

For defects naturally represented by magnitude—image bytes, DOM nodes, JavaScript CPU time, request count, artificial latency, or injected shift magnitude—several ordered perturbation levels can reveal:

$$\text{profiler response} = f(\text{known perturbation magnitude})$$

The appropriate question is then not only whether the treatment differs from baseline, but whether the profiler is monotonic, saturating, noisy near detection thresholds, or insensitive over part of the range. Long tasks are a particularly natural example because the Web platform itself gives the construct a 50 ms reporting boundary. [16]

This need not imply an arbitrary number of dosage levels; their number and spacing should follow the precision and functional behavior you need to characterize.

Multiple baseline archetypes can reduce fixture-specific conclusions

One ultra-simple baseline provides excellent control but can make conclusions dependent on an artificial page architecture. Current browser benchmark design illustrates an alternative: Speedometer uses multiple application patterns and implementations so that browser responsiveness is not represented by one micro-workload alone. [8]

Thus one reasonable option is to maintain a **small family of baseline archetypes**, such as a document-oriented page and an interactive application, and apply the same conceptual perturbations where they make sense. This is not necessary for every causal question, but it provides a way to distinguish "the profiler detects this effect" from "the profiler detects it only in this specific fixture implementation."

A useful additional condition is a **sham perturbation**: rebuild/redeploy the fixture through the same pipeline without changing the performance factor. It can reveal effects of asset fingerprinting, bundling, server restart, caches, or deployment mechanics that might otherwise masquerade as the treatment.

Finally, because the profiler itself may alter execution, a separate **instrumentation-overhead experiment** should compare equivalent executions with and without profiling enabled. Performance research has repeatedly demonstrated that seemingly innocuous experimental setup differences can affect measured program performance. [18]

Live corpus selection and workload operationalization

Selection should begin with the population you intend to describe

The fundamental question is not "How many websites should I choose?" but:

> **What population would a result over these websites be intended to describe?**

Different selection methods lead to different legitimate claims.

Corpus strategy	What it does well	What it cannot safely establish by itself
Simple random sampling from a defined frame	Supports inference to that frame when every eligible unit has a known/equal selection mechanism.	It cannot generalize beyond biases in the underlying frame. A CrUX-derived frame, for example, contains sufficiently visited and eligible Chrome destinations rather than every possible Web site. [19]
Stratified random sampling	Ensures representation of important strata—for example popularity bands, application types, or measured resource-intensity bands—and can improve precision for a heterogeneous Web population.	Disproportionate stratum sampling requires weighting if estimating population-wide quantities.
Popularity-based top-N	Targets prominent sites that users are likely to encounter and is common in published Web measurement. Tranco was created partly to make such top-site selections more stable and reproducible. [20]	Does not constitute a random sample of "the Web"; conclusions should be about the selected popularity population.
Random selection within a popularity range	Retains relevance to popular sites while reducing investigator discretion. Klotski, for example, randomly chose pages inside a specified Alexa popularity frame. [6]	Still inherits the top-list frame and its definition of popularity.
Purposive workload categories	Excellent for coverage and stress testing: intentionally include image-dense, JavaScript-intensive, DOM-intensive, third-party-rich, interactive, and low-complexity workloads.	Category proportions do not support prevalence claims unless the selection procedure itself supports them.
Framework/application stratification	Tests robustness across different execution architectures—for example React, Vue, traditional server-rendered documents, media sites, dashboards. Speedometer similarly spans frameworks and workload patterns. [13]	Framework detection is imperfect, and a framework label does not itself establish that JavaScript/DOM/network behavior is heavy. HTTP Archive explicitly warns that technology-detection mechanisms have limitations. [11]
Probability-oriented backbone plus purposive challenge set	Separates broad ecological coverage from deliberately difficult profiler workloads.	Results from the two subsets must remain analytically distinguishable; a challenge set should not silently be folded into population estimates.

For a profiler evaluation, the last approach is particularly attractive as a **design option**: a reproducibly sampled live corpus can answer robustness/generalization questions, while a separately identified challenge subset can ensure that relatively uncommon but important workload structures are exercised. The two sets need not support identical claims.

Freeze the sampling frame

Popularity rankings themselves evolve. Tranco was explicitly designed to improve robustness and reproducibility of top-site studies and provides archived, permanently referenceable rankings with their methodology preserved. [20]

Thus, if a top-site list is used, record the **exact list snapshot or permanent identifier**, not merely "the Tranco top sites." Likewise, a CrUX-derived frame should be tied to a specific release period because CrUX eligibility and origins change over time; CrUX describes itself as representing real Chrome users at eligible popular Web destinations rather than all pages on the Web. [21]

Sample pages, not merely domain names, when page behavior is the unit of interest

A site homepage is not automatically representative of its internal application behavior. HTTP Archive explicitly identifies homepage-only measurement as a source of bias and has increasingly incorporated secondary pages for this reason. [11]

Consequently, a study should make an explicit choice among:

$$\text{origin/domain} \quad / \quad \text{URL/page} \quad / \quad \text{user journey.}$$

If the profiler is intended primarily for page-load diagnosis, page URLs may be the natural workload unit. If it is intended for application profiling after initial navigation, a URL plus a scripted interaction trace is the more defensible unit. For single-page applications in particular, only measuring the initial document navigation can omit important post-load execution; current Web-performance work likewise distinguishes initial navigation from interaction and soft-navigation behavior. [22]

"Static," "image-heavy," and similar categories should normally be operationalized

For quantitative research, labels such as **image-heavy**, **JavaScript-heavy**, and particularly **poorly optimized** should not depend solely on researcher intuition.

Butkiewicz et al. provide an early precedent by representing Web complexity with measurable content- and service-level features rather than only descriptive website labels. [5]

A possible operational vocabulary is:

Informal label	More reproducible operationalization candidates
Image-heavy	Image transfer bytes; image bytes as fraction of total transfer; image request count; decoded pixel area; number of above-the-fold images.

Informal label	More reproducible operationalization candidates
JavaScript-heavy	JavaScript transfer bytes; number of scripts; parse/compile/evaluation CPU; total main-thread JavaScript time; number or total duration of ≥50 ms long tasks. The last measure has an externally specified threshold through the Long Tasks API. [16]
DOM-heavy	DOM node count, tree depth, maximum branching, elements participating in layout/style recalculation.
Request-heavy	Number of network requests, potentially separated into first-party/third-party and by resource type or origin.
Third-party-heavy	Number of third-party origins, requests and transfer bytes, plus main-thread time attributable to third-party scripts where attribution is possible.
Layout-unstable	Number and magnitude of layout-shift events or CLS under a precisely specified load/interaction trace. CLS is based on unexpected visual movement and therefore has a concrete performance meaning, unlike a subjective "unstable" label. [17]
Static	Prefer a measurable label such as **low-dynamicity**: minimal JavaScript execution, minimal post-navigation DOM mutation, few/no asynchronous fetches after the designated load point, and no interaction-required rendering during the trace. "Static" is otherwise ambiguous because a server-generated document may still load substantial client-side behavior.
Poorly optimized	Prefer not to use as a primitive workload class. Replace it with explicit attributes such as oversized image payload, lack of compression, render-blocking resources, ineffective caching, excessive transfer, or high unused-script cost.

There are two good ways to turn continuous properties into categories. One is to predefine **absolute criteria** where an external specification gives the construct a meaningful boundary—for example the W3C long-task threshold. [16] The other is to derive relative groups from the sampling frame—for example upper/lower quantiles of JavaScript CPU or image-byte fraction. In the second case the claim becomes "high relative to this corpus," not universally "heavy."

An especially important guard against circularity is:

> **Do not define a live-site category using the custom profiler output and then cite the profiler's differing output across those same categories as validation.**

Use independent browser traces, resource metadata, standardized APIs, or source/network measurements to define the workload characteristics whenever the category will subsequently be used to evaluate the profiler.

"Poorly optimized" deserves special caution because it mixes an outcome judgment with assumptions about causality. A page may be large or slow for legitimate functional reasons. Core Web Vitals such as CLS are useful outcome measures, but a poor outcome does not by itself prove which engineering optimization is

missing. For example, CLS measures visual instability; common causes include unsized media and dynamically injected content, but the metric itself is not a diagnosis of the implementation defect. [23]

Corpus size should emerge from the inferential target, not convention

The literature uses corpus sizes from dozens of pages to hundreds, thousands, or millions because the studies address very different questions; WProf used hundreds, Jueckstock et al. used a top-25K frame, and HTTP Archive operates at multi-million-site scale. Those numbers are precedents, **not sample-size justifications for your study**. [24]

An adequate number of live sites depends on at least:

$$N_{\text{sites}} = f(\text{desired precision}, \text{between-site variance}, \text{target effect}, \text{strata}, \text{interactions}, \text{attrition}, \text{estimand}).$$

A defensible process is therefore to obtain pilot measurements, estimate variance at different levels, and then choose the site count from a desired confidence-interval width or statistical power for the actual model. Kalibera and Jones' systems-benchmarking methodology specifically argues for determining repetitions from measurement variation and desired precision instead of defaulting to arbitrary repetition counts, and for reporting effect-size uncertainty. [25]

Crucially, **more repeated loads of the same website do not substitute for additional independent websites when the intended inference concerns websites**. Additional runs improve the estimate of that site's condition mean; additional sites provide information about between-site heterogeneity and population generalization. That distinction is central to avoiding pseudoreplication.

Experimental-variable map and repeated-measures structure

Candidate variable map

The following is a map of possible roles rather than a declaration that every variable must occupy that role in your eventual design.

Role	Candidate variables	Interpretation
Independent variables — controlled fixture tier	Perturbation type; perturbation magnitude; fixture archetype; agent profile; profiler enabled/disabled; intentionally manipulated cache/network/CPU configuration.	Variables deliberately set by the experimenter.
Independent variables — live tier	Agent profile; profiler configuration/version; intentionally assigned cache policy or vantage point.	Variables you can genuinely manipulate on an otherwise observational page.
Observed predictors / corpus attributes	Image-byte fraction, JS CPU, DOM size, request count, third-party count, framework, popularity band, application category.	These describe live pages but are generally not experimental treatments.

Role	Candidate variables	Interpretation
Dependent variables — profiler behavior	Reported bottleneck type; attributed duration; ranking of bottlenecks; event/task/resource attribution; generated recommendations; profiler processing time; missing/failed attribution.	Direct outputs of the system being evaluated.
Dependent variables — independent reference measurements	Navigation/load metrics, browser trace durations, resource timings, request/byte counts, long-task measurements, layout-shift values, CPU utilization.	Useful for criterion/convergent validation where they do not simply reproduce the profiler's own implementation.
Dependent variables — fixture sensitivity	Difference from baseline, response slope over perturbation magnitude, detection probability, attribution error relative to injected ground truth.	Particularly relevant to the controlled sensitivity tier.
Controlled variables	Browser build, browser flags, OS/runtime image, hardware allocation, viewport/DPR, test interaction trace, intended cache state, profile state, fixture server configuration, test protocol, location when not under study.	Held constant because they are not the target of that comparison.
Nuisance variables	Time of day, server/CDN load, DNS/path changes, third-party variation, live-site deployment changes, ad selection, personalization, bot detection, thermal state, background OS activity, browser-process history, run order.	Can affect outcomes without being the target of inference; control, block, record, or randomize them as feasible.

The controlled-variable versus nuisance-variable distinction is partly operational. A variable becomes “controlled” only if the experiment can actually hold it fixed. CDN backend choice on an external production site, for example, is normally **not controlled merely because the researcher wishes it were constant**; it remains a nuisance source to be recorded, blocked against indirectly, or incorporated into variability.

NIST defines nuisance factors as factors that affect the response but are not themselves of primary experimental interest, and randomized blocking is expressly intended to absorb such variation. [26]

Websites and agent profiles are usually crossed, not nested

The wording “measurements nested across websites and agent profiles” needs one statistical refinement.

Suppose every selected site is tested using every agent profile:

$$\mathrm{Site}_1 \times \{\mathrm{Agent}_1, \mathrm{Agent}_2, \ldots\}$$

$$\mathrm{Site}_2 \times \{\mathrm{Agent}_1, \mathrm{Agent}_2, \ldots\}.$$

Then **site and agent are crossed factors**. Agent is not nested inside website because the same agent levels occur for multiple websites.

Individual execution replicates can instead be viewed as nested within a site × agent × condition cell:

$$y_{s,a,c,b,r},$$

where s denotes site, a agent, c experimental condition, b temporal or execution block, and r the repeated load.

A conceptual mixed-effects representation could be:

$$y_{sacr} = \mu + \alpha_a + \beta_c + (\alpha\beta)_{ac} + u_s + v_{s,c} + \gamma_b + \epsilon_{sacr}.$$

Here, agent and experimental condition might be fixed effects when you care specifically about the chosen profiles/conditions; website can be a random effect when inference is intended to extend beyond the sampled sites; and a site-specific treatment slope $v_{s,c}$ allows the profiler/condition effect to differ by website. This is one modelling option, not a required final model.

That site-specific slope is conceptually important: averaging away heterogeneity could hide a profiler that behaves well for one workload family and poorly for another.

For controlled fixtures, fixture identity or archetype can similarly be treated as fixed when those exact fixtures are themselves the only target, or as a source of variability when they are intended to stand in for a broader class.

Synthetic agent profiles should have explicit scope

Configurable synthetic user-condition agents are methodologically useful, but a synthetic configuration should not automatically be described as a representative “user.”

Jueckstock et al. showed that browser configuration and network vantage point can change observed Web behavior even when other aspects are tightly controlled. Their design compared three vantage points and two browser configurations using synchronized visits, explicitly treating these as realism variables. [10]

Zeber et al. went further by comparing crawls against actual Firefox browsing data and observed differences associated with stateless crawlers versus generally stateful users, as well as differences over time and between cloud and residential execution. [27]

It is therefore more defensible to say that a profile represents, for example:

> “the specified mobile-like CPU/network/browser condition”

rather than:

> “a typical mobile user,”

unless empirical user-population data justify the second statement.

Randomization, factorial expansion, and sample-size logic

Randomization and blocking should target temporal drift

A simple but risky run schedule would be:

> all baselines → all perturbations → all Agent A tests → all Agent B tests.

Any time-dependent machine or Web change then becomes partly aliased with the experimental factor. Systems research has repeatedly shown that apparently minor experimental setup differences can materially alter performance measurements. [18]

A stronger general structure is to form **local comparison blocks** and randomize inside them. For example:

$$\text{block} = \text{website} \times \text{agent profile} \times \text{short time window}.$$

Within such a block, baseline and perturbed conditions can be run in randomized order. NIST's randomized-block methodology is specifically intended to separate a treatment of interest from nuisance variation such as run-to-run or time-of-day effects. [26]

For two conditions, balanced order patterns such as:

$$AB,\ BA,\ AB,\ BA, \ldots$$

can protect against a simple first-versus-second position effect. Random assignment of the order to blocks is preferable to mechanically alternating forever because deterministic ordering itself can align with another periodic source of variation.

With several conditions, randomized order within blocks is often sufficient. Latin-square or related designs are available when treatment-order/position must be balanced systematically against another nuisance factor; NIST describes Latin-square families as efficient blocking structures and emphasizes randomizing run order to the extent allowed by the design. [28]

Cache experiments need isolation, not randomization alone

A cache is stateful. If condition A warms resources used by condition B, running A first changes B's experimental state. Randomization balances this contamination statistically but does not eliminate the mechanism.

Thus cold-cache comparisons should establish the intended initial browser/storage state before **each independent replicate**. Warm-cache tests should explicitly define how the warm state is created. Service workers are themselves persistent registrations with fetch-related behavior, reinforcing the need to specify whether service workers, Cache Storage, cookies, HTTP cache and other site data persist between loads. [29]

This is also why “clear cache” should be operationally defined. Browser HTTP cache, cookies, service workers, DNS, TLS/session state, OS caching, CDN edge caching and origin caches are different layers; a study should only claim to have reset the ones it actually resets.

Live-site conditions should be kept temporally close when making paired comparisons

Live pages evolve over minutes as ads, third-party requests, backend routing, deployments and personalized resources change. Zeber et al. found measurable divergence even among equivalent Web crawls and substantial change with time; in their measurements, specific resource URLs became markedly less similar as crawl timing diverged. [30]

This makes **temporally paired conditions** particularly valuable for live-site comparisons. If Agent A and Agent B are being compared, running them near each other—or, where appropriate and ethically/load-wise feasible, contemporaneously—reduces temporal page evolution as an alternative explanation.

Jueckstock et al. used synchronized parallel visits to create matched observations across their vantage-point/browser-configuration combinations. [31] Parallel execution is therefore an available design option, although it should not be used so aggressively that the experiment itself creates appreciable load on the target site.

Time of day can be blocked rather than eliminated

If production variability is part of the ecological environment you intend to characterize, always running at one carefully chosen quiet hour may improve repeatability but reduce external coverage.

An alternative is to distribute measurement blocks across predefined temporal windows and retain time/block metadata. Then:

$$\text{time variation}$$

is represented deliberately instead of silently confounded with agent or condition.

The right choice depends on the estimand. A tightly controlled profiler comparison might fix a short temporal period; an ecological robustness study might deliberately sample several periods. These are different questions.

Factorial experimentation becomes useful after single-factor behavior is understood

One-factor perturbations are excellent for **mechanistic interpretation**, but real performance defects interact. A fixed JavaScript workload is more damaging under a weak CPU; large image bytes are more costly at low bandwidth; request multiplicity interacts with RTT, protocol and cache state.

A natural second phase is therefore to study selected interactions such as:

$$\text{JavaScript work} \times \text{CPU profile}$$

$$\text{image bytes} \times \text{bandwidth}$$

$$\text{request count} \times \text{RTT}$$

$$\text{request count} \times \text{HTTP protocol}$$

$$\text{cacheability} \times \text{cache state}.$$

In a full factorial design, every level of every factor occurs with every level of the others; for k two-level factors this creates 2^k combinations. [32] Because that grows rapidly, well-chosen fractional factorial designs can retain balanced and orthogonal structure while evaluating only a subset of combinations. [33]

The important sequencing point is that factorial experiments should not replace the interpretable single-factor calibration tier. They answer a different question:

> **Does profiler response remain interpretable when causes interact?**

rather than:

> **Can the profiler detect this known defect in isolation?**

Repetition counts and website counts answer different variance questions

A useful variance decomposition is:

$$\sigma^2_{\text{total}} = \sigma^2_{\text{website}} + \sigma^2_{\text{site}\times\text{agent}} + \sigma^2_{\text{time block}} + \sigma^2_{\text{repeated run}} + \cdots .$$

A pilot can estimate which levels dominate.

If within-cell run noise is large, more repetitions are valuable. If between-site variation dominates and the intended inference concerns sites, allocating essentially all additional budget to repeated runs on a tiny set of sites produces diminishing inferential value. Kalibera and Jones' rigorous benchmarking work is directly relevant here because it treats repetition as a hierarchical measurement-design problem and argues for repetition choices driven by precision and observed variation rather than ritual fixed counts. [25]

The sample-size decision can therefore be based on one of two defensible targets:

$$P(|\hat{\theta} - \theta| < \delta)$$

for a desired estimation precision, or

$$P(\text{detect effect of specified magnitude})$$

for a power-based target.

For a mixed-effects design, simulation based on pilot variance components is often more appropriate than a simple independent-samples formula because websites, agents, conditions and repeated executions are crossed/nested in different ways.

This avoids declaring an arbitrary number of websites now while still providing a pre-specifiable rule for deciding later.

Validity and reproducibility

Internal validity of controlled fixtures

The controlled fixture tier can offer very high internal validity for sensitivity questions because the experimenter controls the causal manipulation and can maintain independent ground truth. That is exactly the benefit of the progression from simple synthetic conditions toward realistic page loads in the SPDY study. [3]

The strongest claim supported by such evidence would have the form:

> Under the specified browser, agent and fixture conditions, deliberately increasing factor X while holding specified aspects constant produces the observed change in profiler output.

That is much stronger causally than observing that naturally JavaScript-heavy production sites happen to receive higher profiler scores.

Key internal-validity threats remain:

Threat	Consequence	Mitigation option
Perturbation coupling	Intended “image-size” change also changes format, dimensions, DOM or request count.	Store factor manifests and verify non-target properties automatically.
Unrealistic implementation	An artificial defect exercises a code path unlike real pages.	Use several fixture archetypes and later test live or recorded pages.
Order/carryover	Cache, JIT, browser process, or thermal state depends on previous run.	Isolation, blocking, randomization and explicit state reset.
Measurement reactivity	Profiler instrumentation changes the timing it measures.	Profiler-off/on overhead experiments.
Fixture-specific optimization	Browser happens to optimize the particular synthetic structure unusually well.	Multiple dose levels and, where useful, multiple baseline architectures.
Hidden deployment change	Rebuild generates different hashes/ bundles independent of treatment.	Sham treatment plus artifact comparison.
Ground truth derived from profiler	Evaluation becomes circular.	Independent instrumentation or exact source-level quantities.

These controls are consistent with the broader systems-performance finding that seemingly benign environmental or layout changes can alter benchmark outcomes. [18]

External and ecological validity of live sites

Live production pages restore precisely the complexity removed by the fixtures: actual network paths, DNS and CDN behavior, third-party services, asynchronous variation, production application code and changing backend responses.

HTTP Archive's methodology illustrates both sides of this tradeoff. It tests millions of real websites, but explicitly describes itself as **lab testing** because its browser agents execute from datacenters under predefined configurations, with empty caches and logged-out profiles; the methodology warns that these conditions may differ from real user experience. [11]

Zeber et al. provide direct empirical evidence for that warning: automated crawl environments and real Firefox browsing differed, with browser history/state, operating system, network environment, geographic location and differing portions of sites all identified as sources of representativeness concerns. [27]

Consequently, live evaluation supports progressively stronger claims depending on its sampling design:

Purposive corpus:
“The profiler successfully executed and produced interpretable results over these deliberately diverse live workloads.”

Popularity-based corpus:
“The profiler was evaluated over the selected population of popular sites as defined by the frozen ranking.”

Probability sample from a defined frame:
“With the stated uncertainty and inclusion criteria, results can be generalized to the defined sampling frame.”

It should generally **not** support a statement such as “this represents ordinary users' Web experience” merely because URLs were live.

Synthetic network/device agents improve scenario coverage but do not eliminate this distinction. Jueckstock et al. found measurable effects from both network vantage point and browser configuration, while Zeber et al. found differences between stateless crawl infrastructure and real stateful browsing. [34]

The hybrid design supports triangulation

The two corpus tiers can be viewed as addressing different validity dimensions:

Controlled fixtures	Live production corpus
Known causal manipulation	Naturally occurring complexity
Known ground truth	Usually no known causal ground truth

Controlled fixtures	Live production corpus
Strong reproducibility	Site can change or disappear
Strong internal validity	Stronger ecological realism
Limited natural diversity	Large architectural diversity
Can isolate defect sensitivity	Can expose robustness/failure modes
Artificial infrastructure if locally hosted	Actual CDN/server/network/third-party behavior

A third, optional **record-and-replay layer** can bridge the two. Mahimahi's explicit purpose is to record traffic from HTTP applications and replay it under emulated networks while retaining realistic multi-server characteristics. [4] WProf likewise used locally hosted copies for some controlled experiments, although its authors noted fidelity problems for dynamically generated objects that could not be captured. [35]

Thus the evidence can be conceptualized as a continuum:

$$\text{fully synthetic} \rightarrow \text{controlled real-derived replay} \rightarrow \text{live production Web}.$$

You do not necessarily need all three, but existing systems research clearly supports moving along this control–realism continuum. [36]

Reproducibility requirements for the controlled corpus

Each fixture should be reconstructable independently of the live Web. At minimum, preserve:

Artifact	Recommended provenance
Source	Repository commit/tag for baseline and every perturbation.
Dependency state	Lockfiles and vendored or content-addressed dependencies where practical.
Generated assets	Cryptographic hashes or immutable artifact bundle.
Perturbation definition	Factor, level, changed files/resources and ground-truth quantities.
Server	Server software/version/configuration, HTTP/TLS configuration, certificates if relevant.
Build environment	Toolchain versions and preferably a container/VM or equivalent machine-readable environment.
Random generation	Seeds for any fixture generator.
Interaction trace	Exact script, event sequence, timings and completion criteria.
Expected state	Assertions such as exact request count, image bytes, DOM count or injected task description.

A fixture that imports mutable third-party dependencies at test time substantially weakens the “controlled” argument, because it makes the workload capable of changing independently of your versioned source.

Reproducibility requirements for the execution environment

Browser benchmarking practice already treats seemingly mundane client state as important. Speedometer advises a clean browser profile, no background applications, closing other tabs/windows, keeping the benchmark focused, plugging in battery-powered devices, and accounting for device cooling between runs. [37]

For profiler experiments, the execution manifest should be more detailed:

Browser: exact version/build, executable hash where practical, command-line flags, headless/headful state, extensions, feature flags.

Machine: hardware model, CPU/memory allocation, OS/kernel, virtualization/containerization, power mode and any CPU-frequency controls that materially affect execution.

Display/client: viewport, device-pixel ratio, user-agent string, locale/timezone and accessibility settings if they can alter content.

Browser state: clean/new versus persistent profile, cookies, HTTP cache, Cache Storage, local/session storage, service workers, permission state.

Agent: CPU throttling method, network bandwidth/RTT/loss configuration, geographic/network vantage point, DNS configuration.

Test procedure: navigation start definition, completion condition, interaction sequence, timeout behavior, retry policy and exclusion rules.

Profiler: exact profiler commit/version, configuration, enabled modules, sampling period, collection options and post-processing version.

Jueckstock et al. give a useful exemplar: their paper explicitly documented clean profiles, timeouts, common DNS, deterministic handling of JavaScript entropy sources, bandwidth/latency controls, network vantage point and browser configuration. [31]

Reproducibility requirements for raw data

Preserve the data **before** aggregation or exclusion wherever storage permits:

$$\text{raw run} \rightarrow \text{validated run} \rightarrow \text{derived metrics} \rightarrow \text{statistical summary}.$$

Useful raw artifacts include profiler traces, browser traces, HAR/network logs, resource metadata, console/runtime errors, navigation timing, long-task/layout-shift events, screenshots when they help establish page state, run timestamps, agent metadata and exit/failure status.

Exclusions should not silently delete data. A timeout, bot-blocking page, consent wall, DNS failure, redirect loop or profiler crash is itself part of the measurement record. The exclusion rule can later determine whether it contributes to the primary analysis.

Reproducibility requirements for live URLs

A list of URLs alone is not a reproducible corpus. The Web changes continuously; Zeber et al. directly observed changing loaded resources over time, including substantial resource-URL divergence across measurement periods. [30]

For every live measurement, preserve at least:

$$\{\text{requested URL}, \text{final URL}, \text{timestamp}, \text{list snapshot}, \text{agent}, \text{resource manifest}, \text{response metadata}\}.$$

Where legally and ethically appropriate, also preserve the HTML/resource responses or a standards-compatible archival capture, plus page screenshots and hashes. Captures cannot perfectly reproduce a dynamic production service, but they document what the browser encountered considerably better than the URL alone.

Recent Web-measurement work makes this point particularly strongly. Hantke et al.'s **WebREC / Web Execution Bundles** work proposes an archival format specifically designed to preserve Web execution information for reproducible measurement and argues that conventional archives or replay in a different-era browser can change the behavior being measured. [38]

That creates an important distinction:

$$\text{reproduce the historical bytes} \;/\; \text{reproduce the historical browser execution}.$$

For performance experiments that may be revisited years later, preserving both **Web content provenance and the original browser environment** is therefore preferable.

Priority original papers to download

The following would form a strong methodological reading set. The priority reflects relevance to your experimental design, not a judgment about paper quality.

Priority	Original work	Why it is especially relevant
Highest	**Xiao Sophia Wang et al., "Demystifying Page Load Performance with WProf," NSDI 2013.** [39]	Closest direct analogue: evaluation of an in-browser Web-performance profiler, use of hundreds of real pages, repeated measurements and controlled experiments.

Priority	Original work	Why it is especially relevant
Highest	**Xiao Sophia Wang et al., “How Speedy is SPDY?”, NSDI 2014.** [3]	Probably the most useful precedent for your proposed experimental logic because it explicitly moves from simple synthetic pages through isolated factors to complete real-derived page loads.
Highest	**Ravi Netravali et al., “Mahimahi: Accurate Record-and-Replay for HTTP,” USENIX ATC 2015.** [4]	Core reference for the middle ground between controlled fixtures and live URLs: production Web capture plus reproducible network replay.
Highest	**Michael Butkiewicz, Harsha V. Madhyastha, and Vyas Sekar, “Understanding Website Complexity: Measurements, Metrics, and Implications,” IMC 2011.** [5]	Especially valuable for operationalizing “heavy” workload classes using quantitative content/service metrics.
Highest	**Jordan Jueckstock et al., “Towards Realistic and Reproducible Web Crawl Measurements,” WWW 2021.** [40]	Directly relevant to agent profiles, browser configuration, network vantage point, confound control, synchronized visits and documenting a live-Web crawl.
Highest	**David Zeber et al., “The Representativeness of Automated Web Crawls as a Surrogate for Human Browsing,” WWW 2020.** [41]	Essential external-validity reading. Empirically tests when automated visits differ from actual human browsing.
Highest	**Tomas Kalibera and Richard E. Jones, “Rigorous Benchmarking in Reasonable Time,” ISMM 2013.** [25]	Particularly useful for repetition strategy, hierarchical sources of variation, confidence intervals and avoiding arbitrary repetition counts.
High	**Todd Mytkowicz et al., “Producing Wrong Data Without Doing Anything Obviously Wrong!”, ASPLOS 2009.** [42]	Classic systems-performance warning about apparently innocuous experimental setup influencing measured performance and conclusions.
High	**Andy Georges, Dries Buytaert, and Lieven Eeckhout, “Statistically Rigorous Java Performance Evaluation,” OOPSLA 2007.** [43]	Useful general foundation for repetitions, distributions, statistical comparison and confidence in software-performance experiments.

Priority	Original work	Why it is especially relevant
High	**Victor Le Pochat et al., "Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation," NDSS 2019.** The accompanying service archives exact ranking snapshots for reproducibility. [20]	Important if any part of your live sampling frame is popularity-based.
High	**Nadine Demir et al., "Reproducibility and Replicability of Web Measurement Studies," WWW 2022.** [44]	Detailed guidance on crawler/browser state, geolocation, browser configuration and release of measurement artifacts. Particularly useful when drafting the methodology checklist.
High	**Florian Hantke et al., "Web Execution Bundles: Reproducible, Accurate, and Archivable Web Measurements," USENIX Security 2025.** [45]	Current reference for preserving dynamic Web measurements and understanding why conventional Web archives do not necessarily reproduce original browser execution.
Supporting	**Butkiewicz et al., "Klotski: Reprioritizing Web Content to Improve User Experience on Mobile Devices," NSDI 2015.** [6]	Concrete precedent for random site selection inside a popularity frame and mobile Web-performance evaluation.
Supporting	**Netravali et al., "Polaris: Faster Page Loads Using Fine-Grained Dependency Tracking," NSDI 2016.** [7]	Useful example of a recorded popular-site corpus spanning application categories.
Benchmark artifact rather than paper	**Speedometer 3.1 documentation and benchmark source/design.** [46]	Valuable current example of controlled browser workload construction, reproducible user journeys, framework diversity and environmental test guidance.
Large-scale methodology reference rather than research paper	**HTTP Archive / Web Almanac 2025 methodology.** [11]	Particularly useful for documenting sampling frames, page versus homepage bias, desktop/mobile agents, cold-cache assumptions, datacenter limitations and reproducible queries.

Taken together, these sources support a methodological interpretation in which the **controlled and live corpora serve intentionally different inferential purposes**. The controlled tier can provide traceable causal perturbations, dose-response evidence and independent ground truth; the live tier can expose the profiler to real CDN/network/server/third-party/application variability; and record/replay is an optional intermediate mechanism when production-derived complexity must be made reproducible. [47]

The central design constraint is consequently not choosing one corpus over the other, but preserving the boundary between the claims each corpus can justify: **causal sensitivity should come primarily from deliberate manipulation; population/ecological claims should come from the sampling design and live execution environment; and repeated runs should quantify measurement variability without being mistaken for independent website replication.** [48]

---

[1] [2] [12] [24] [39] Demystifying Page Load Performance with WProf
https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/wang_xiao?utm_source=chatgpt.com

[3] [15] [36] [47] How Speedy is SPDY?
https://www.usenix.org/conference/nsdi14/technical-sessions/wang?utm_source=chatgpt.com

[4] [14] Mahimahi: Accurate Record-and-Replay for HTTP
https://www.usenix.org/conference/atc15/technical-session/presentation/netravali?utm_source=chatgpt.com

[5] Understanding website complexity: measurements, metrics ...
https://dl.acm.org/doi/10.1145/2068816.2068846?utm_source=chatgpt.com

[6] Klotski: Reprioritizing Web Content to Improve User ...
https://www.usenix.org/system/files/conference/nsdi15/nsdi15-paper-butkiewicz.pdf?utm_source=chatgpt.com

[7] Polaris: Faster Page Loads Using Fine-grained ...
https://www.usenix.org/system/files/conference/nsdi16/nsdi16-paper-netravali.pdf?utm_source=chatgpt.com

[8] [13] [46] Speedometer 3.1 About
https://browserbench.org/Speedometer3.1/about.html?utm_source=chatgpt.com

[9] [27] [30] The Representativeness of Automated Web Crawls as a Surrogate for Human Browsing
https://research.mozilla.org/files/2020/02/Jestr_vs_crawl_WWW20202.pdf

[10] [31] [34] Towards Realistic and ReproducibleWeb Crawl Measurements
https://www.kapravelos.com/publications/vpc-www21.pdf

[11] Methodology | The Web Almanac by HTTP Archive
https://almanac.httparchive.org/en/2025/methodology?utm_source=chatgpt.com

[16] Long Tasks API
https://www.w3.org/TR/longtasks-1/?utm_source=chatgpt.com

[17] [23] Cumulative Layout Shift (CLS) | Articles
https://web.dev/articles/cls?utm_source=chatgpt.com

[18] [42] Producing wrong data without doing anything obviously ...
https://dl.acm.org/doi/10.1145/1508244.1508275?utm_source=chatgpt.com

[19] [21] Overview of CrUX | Chrome UX Report - Chrome for Developers
https://developer.chrome.com/docs/crux?utm_source=chatgpt.com

[20] Tranco: A research-oriented top sites ranking hardened ...
https://tranco-list.eu/?utm_source=chatgpt.com

[22] Measuring soft navigations | Web Platform
https://developer.chrome.com/docs/web-platform/soft-navigations?utm_source=chatgpt.com

[25] Rigorous benchmarking in reasonable time
https://dl.acm.org/doi/10.1145/2491894.2464160?utm_source=chatgpt.com

[26] [48] 5.3.3.2. Randomized block designs
https://www.itl.nist.gov/div898/handbook/pri/section3/pri332.htm?utm_source=chatgpt.com

[28] 5.3.3.2.2. Graeco-Latin square designs
https://www.itl.nist.gov/div898/handbook/pri/section3/pri3322.htm?utm_source=chatgpt.com

[29] Service Workers Nightly
https://www.w3.org/TR/service-workers/?utm_source=chatgpt.com

[32] 5.3.3.3. Full factorial designs
https://www.itl.nist.gov/div898/handbook/pri/section3/pri333.htm?utm_source=chatgpt.com

[33] 5.3.3.4. Fractional factorial designs
https://www.itl.nist.gov/div898/handbook/pri/section3/pri334.htm?utm_source=chatgpt.com

[35] usenix.org
https://www.usenix.org/system/files/conference/nsdi13/nsdi13-final177.pdf

[37] Speedometer 3.1 Test Instructions
https://browserbench.org/Speedometer3.1/instructions.html?utm_source=chatgpt.com

[38] [45] usenix.org
https://www.usenix.org/system/files/usenixsecurity25-hantke.pdf

[40] Towards Realistic and ReproducibleWeb Crawl ...
https://dl.acm.org/doi/10.1145/3442381.3450050?utm_source=chatgpt.com

[41] The Representativeness of Automated Web Crawls as a ...
https://dl.acm.org/doi/10.1145/3366423.3380104?utm_source=chatgpt.com

[43] Statistically rigorous java performance evaluation
https://dl.acm.org/doi/10.1145/1297027.1297033?utm_source=chatgpt.com

[44] Reproducibility and Replicability of Web Measurement Studies
https://intellisec.de/pubs/2022-www.pdf?utm_source=chatgpt.com